Review Sentiment Analysis: Project Walkthrough and Presentation Script

Presenter guide | Based on the repository implementation

1 November 2026

How to present this project

This guide follows the order in which the system runs: get the data, validate and split it, train the model, evaluate and accept it, serve predictions, then monitor and deploy. The **Say** paragraphs are a script you can read aloud. The notes after each one give technical detail for follow-up questions.

1. Introduce the project

Say: “This is an end-to-end MLOps project that classifies a review sentence as positive or negative. It covers more than model training. It downloads and validates public data, trains and evaluates a model, saves the model only if it passes a quality gate, serves it through an API with a web interface, and adds monitoring, container deployment, and automation.”

The main path is:

`UCI dataset → combine + validate → stratified split → TF-IDF + Logistic Regression (fixed settings) → evaluation + acceptance gate → model artifact → FastAPI → web page + Prometheus/Grafana`

2. Step 1: Fetch the data

Say: “The project downloads the UCI Sentiment Labelled Sentences dataset. It has 3,000 sentences: 1,000 each from Amazon product reviews, Yelp restaurant reviews, and IMDB movie reviews. Each sentence is labelled 1 for positive or 0 for negative, with exactly 500 of each per source, so the classes are balanced.”

The archive contains three tab-separated files. `download_data.py` reads all three, adds a `source` column, and writes a single file, `data/sentiment_sentences.csv`. The files are read with quoting disabled because the sentences contain bare quotation marks. If a download fails, the program raises an error instead of inventing fallback data.

3. Step 2: Clean and validate

Say: “The loader maps the labels to `positive` and `negative`, drops empty sentences, removes exact duplicates, stops with an error if the same sentence appears with two different

labels, and checks that both classes are present."

The current release has 18 duplicate sentences, leaving 2,982 records (1,490 positive and 1,492 negative). The loader records the record count, duplicate count, label distribution, and source distribution in the training report.

4. Step 3: Split the data

Say: "Before training, 20 percent of the data is set aside as a test set. The split is stratified, so both parts keep the same positive/negative balance, and it uses a fixed seed so results can be reproduced."

`train_test_split(..., test_size=0.2, random_state=42, stratify=labels)` gives 2,385 training rows and 597 test rows.

5. Step 4: Explain the model

Say: "The model has two stages. TF-IDF turns each sentence into numbers: words and two-word phrases get weights, and words that appear everywhere count less. Logistic Regression then learns a weight for each feature: positive weights push toward positive sentiment and negative weights push toward negative sentiment."

Setting	Value	Why
`ngram_range`	`(1, 2)`	Bigrams capture phrases such as "not good" and "waste of"
`sublinear_tf`	`True`	A word repeated many times gets diminishing extra weight
Stop words	not removed	Words such as "not" and "no" carry sentiment
`C`	`1.0`	Default regularization strength
`max_iter`	`1000`	Gives the optimizer room to converge

Say: "This project deliberately does no hyperparameter tuning. The settings are fixed and written in the code, so training is a single fit that takes about a tenth of a second."

Because the vectorizer and the classifier are saved together as one pipeline, the same transformation is used at training time and at prediction time.

6. Step 5: Evaluate on the held-out test set

Say: “The trained model predicts the untouched test set once. We report accuracy, precision, recall, and F1 for the positive class, F1 for the negative class, macro F1, and a confusion matrix.”

Reference result: **accuracy 0.792, macro F1 0.792**. The confusion matrix, in the order negative then positive, is `[[234, 65], [59, 239]]`. Because the classes are balanced, accuracy and macro F1 are almost identical. Macro F1 is still used as the gate because it treats both classes equally.

7. Step 6: Apply the acceptance rule and save

Say: “Evaluation also works as a release gate. The model is accepted only if its macro F1 is at least 0.75. Every run writes an evaluation report. Only an accepted run replaces the model file. A rejected run raises an error and leaves the previous model untouched.”

Outputs:

- `reports/latest_evaluation.json`: written on every run.
- `artifacts/sentiment_pipeline.joblib`: written to a temporary file, then renamed, so the swap is atomic. Written only when the model is accepted.
- `artifacts/model_metadata.json`: the accepted model’s parameters, metrics, data counts, and library versions.

8. Step 7: Serve predictions

Say: “When the API starts, it loads the saved pipeline. `POST /predict` checks that the text is not blank and not too long, then returns the label, the probability of positive sentiment, and the words that pushed the prediction each way.”

Each word’s contribution is its TF-IDF value in the sentence multiplied by its Logistic Regression coefficient. For “Worst phone I have ever bought, total waste of money”, the strongest negative drivers are *worst*, *waste*, and *money*. Other endpoints are `/model` (the model card shown on the web page), `/health`, `/ready`, `/metrics`, and `/docs`.

9. Step 8: The web interface

Say: “The web page is plain HTML, CSS, and JavaScript, served by the same FastAPI app. You type a review, click Analyze or press Ctrl+Enter, and see the verdict, a meter from negative to positive, the driver words, a model card with test metrics, and a short history of what you analyzed in this session.”

10. Step 9: Docker, monitoring, and automation

Say: “The Dockerfile has three stages. A base stage installs the locked dependencies. A trainer stage downloads the data and trains the model, and the build fails if the model doesn’t pass the gate. The runtime stage contains only the code, the web page, and the accepted model, and

runs as a non-root user. A single `docker compose up --build` starts the API, Prometheus, and Grafana."

Prometheus collects request counts, latency, predictions per class, prediction confidence, failures, and the accepted model's offline metrics every 15 seconds. Grafana loads a ready-made dashboard. CI runs lint and tests, builds the image, and calls `/ready` and `/predict` inside the container. A separate workflow can retrain on demand or once a month and uploads the evaluation report. It never deploys automatically.

11. Limitations and closing

Say: "The model learns from 3,000 short English sentences in three domains, so sarcasm, slang, long texts, and other domains will lower its accuracy. There is no neutral class: mixed sentences land near 50 percent. Traffic metrics show how the service is behaving, not whether its predictions are correct."

Closing summary: "The key idea is the complete path from public data to a monitored prediction service. The data is validated, the model is trained with transparent fixed settings, a held-out test set and a threshold decide whether the model ships, and the same saved pipeline powers the API, the web page, and the metrics."

Quick Q&A prompts

Why no hyperparameter tuning? It keeps the pipeline simple, fast, and fully transparent. The fixed settings are sensible defaults for short text, and they already clear the acceptance gate.

Why Logistic Regression? It is fast and works well with sparse TF-IDF features. It also produces probabilities, and its coefficients make predictions explainable.

Why keep stop words? Negations such as "not" change the sentiment, and bigrams like "not good" depend on them.

Why stratify? So both splits keep the 50/50 class balance.

What happens if a candidate fails the gate? The report is written, training raises an error, and the old model stays. In Docker, the image build fails.